



T003.Q29.Tuan3.TailOp.Cau1-DSLK

Chèn node vào DSKL đơn tang đơn

Bài toán: Cho một danh sách liên kết đơn gồm N số đã có thứ tự tăng dần và một số nguyên X. Hãy chèn một node có giá trị X vào danh sách sao cho danh sách vẫn bảo đảm thứ tự tăng dần.

Input:

- Dòng đầu tiên chứa số nguyên N ($1 \leq N \leq 100$) là số lượng phần tử trong DSKL.
- N dòng tiếp theo, mỗi dòng chứa một số nguyên (theo thứ tự tăng dần).
- Dòng cuối cùng chứa số nguyên X, là giá trị cần chèn vào DSKL.

Output:

- In ra các số trong DSKL sau khi chèn trên một dòng, mỗi số cách nhau một khoảng trắng.

Ví dụ:

INPUT	OUTPUT
5	
1	
3	
4	1 3 4 5 6 7
6	
7	
5	

T003.Q29.BTT3.Lop1

10 problems with a total score of 10

#	Problem	Score
1	T003.Q29.Tuan3.TailOp.Cau1-DSLK	1
2	T003.Q29.Tuan3.TailOp.Cau2-DSLK	1
3	T003.Q29.Tuan3.TailOp.Cau3-STACK	1
4	T003.Q29.Tuan3.TailOp.Cau4-STACK	1
5	T003.Q29.Tuan3.TailOp.Cau5-ShellSort(template)	1
6	T003.Q29.Tuan3.TailOp.Cau6-HeapSort(template)	1
7	T003.Q29.Tuan3.TailOp.Cau7-QuickSort(template)	1
8	T003.Q29.Tuan3.TailOp.Cau8-BinaryInsertionSort(template)	1
9	T003.Q29.Tuan3.TailOp.Cau9-RadixSort(template)	1
10	T003.Q29.Tuan3.TailOp.Cau10-InterchangeSort(template)	1



T003.Q29.Tuan3.TailOp.Cau2-DSLK

Đảo ngược danh sách liên kết đơn

Cho danh sách liên kết chứa N phần tử. Bạn hãy đảo ngược danh sách liên kết này và in danh sách trên một dòng.

Input:

- Dòng đầu tiên chứa số nguyên N là số lượng phần tử trong danh sách liên kết.
- N dòng tiếp theo, mỗi dòng chứa một số nguyên là các phần tử theo thứ tự trong DSKL.

Output:

- Một dòng gồm N số nguyên là các phần tử sau khi đã được đảo ngược.

Ví dụ:

Input	Output
4	
2	
4	7 5 4 2
5	
7	

T003.Q29.BTT3.Lop1

10 problems with a total score of 10

#	Problem	Score
1	T003.Q29.Tuan3.TailOp.Cau1-DSLK	1
2	T003.Q29.Tuan3.TailOp.Cau2-DSLK	1
3	T003.Q29.Tuan3.TailOp.Cau3-STACK	1
4	T003.Q29.Tuan3.TailOp.Cau4-STACK	1
5	T003.Q29.Tuan3.TailOp.Cau5-ShellSort(template)	1
6	T003.Q29.Tuan3.TailOp.Cau6-HeapSort(template)	1
7	T003.Q29.Tuan3.TailOp.Cau7-QuickSort(template)	1
8	T003.Q29.Tuan3.TailOp.Cau8-BinaryInsertionSort(template)	1
9	T003.Q29.Tuan3.TailOp.Cau9-RadixSort(template)	1
10	T003.Q29.Tuan3.TailOp.Cau10-InterchangeSort(template)	1

Chuyen doi co so (su dung ngan xep)

Bạn hãy viết một chương trình chuyển đổi một số N trong hệ cơ số A sang một số ở hệ cơ số B (sử dụng ngăn xếp).

Biết rằng để chuyển đổi số N từ hệ cơ số A sang hệ cơ số B, ta có thể chuyển đổi số N từ hệ cơ số A sang hệ cơ số 10, rồi từ đó chuyển sang hệ cơ số B.

Input:

- Dòng đầu tiên chứa số nguyên T, là số lượng bộ câu hỏi.
- T dòng tiếp theo, mỗi dòng gồm 3 số N, A, B ($2 \leq A, B \leq 16$) trong đó N là số ở hệ cơ số A cần chuyển đổi sang hệ cơ số B.

Output:

- Gồm T dòng, mỗi dòng là số sau khi được chuyển đổi hệ cơ số.

Ví dụ:

Input	Output
4	102222
22B 12 3	15
1111 2 10	101010111100001000110100
ABC234 16 2	1BB
12323 4 16	

T003.Q29.BTT3.Lop1

10 problems with a total score of 10

#	Problem	Score
1	T003.Q29.Tuan3.TailOp.Cau1-DSLK	1
2	T003.Q29.Tuan3.TailOp.Cau2-DSLK	1
3	T003.Q29.Tuan3.TailOp.Cau3-STACK	1
4	T003.Q29.Tuan3.TailOp.Cau4-STACK	1
5	T003.Q29.Tuan3.TailOp.Cau5-ShellSort(template)	1
6	T003.Q29.Tuan3.TailOp.Cau6-HeapSort(template)	1
7	T003.Q29.Tuan3.TailOp.Cau7-QuickSort(template)	1
8	T003.Q29.Tuan3.TailOp.Cau8-BinaryInsertionSort(template)	1
9	T003.Q29.Tuan3.TailOp.Cau9-RadixSort(template)	1
10	T003.Q29.Tuan3.TailOp.Cau10-InterchangeSort(template)	1

Đa thức một biến

Định nghĩa đa thức một biến $f(x)$ có dạng tổng các đơn thức hệ số $x^{\text{so_mu}}$.

Viết chương trình nhập xuất đa thức một biến $f(x)$. Sau đó nhập giá trị biến x và tính giá trị $f(x)$ với kết quả hiện thị chính xác 2 số sau dấu thập phân.

INPUT

Đưa các số trong đó:

- Số nguyên đầu tiên ($0 \leq n \leq 100$): số lượng đơn thức trong đa thức.
- n cặp số tiếp theo, mỗi cặp là một đơn thức gồm: hệ số (số thực) và số mũ (số nguyên ≥ 0).
- Các đơn thức được nhập theo thứ tự giảm dần của số mũ, không có hai đơn thức trùng số mũ.

OUTPUT

In ra 2 dòng như sau:

- Đa thức vừa nhập là: <dạng đa thức>
- Với $x = \text{giá trị } x$, giá trị đa thức là: < $f(x)$ > (giá trị $f(x)$ in đúng 2 chu số thập phân)

Lưu ý xuất đa thức:

- Bien trong đa thức ký hiệu là x .
- Số mũ ký hiệu $^{\text{so_mu}}$.
- Phép nhân không ghi ký hiệu.
- Các ký tự biểu diễn đa thức ghi liên nhau (không khoảng trắng).
- Đơn thức đầu tiên nếu hệ số dương thì không in dấu $+$.
- Đơn thức có hệ số bằng 0 thì bỏ qua, nếu tất cả bằng 0 thì đa thức là 0.
- Đơn thức có hệ số bằng 1 hoặc -1 thì không in số 1 (trừ trường hợp số mũ = 0).
- Đơn thức có số mũ bằng 0 thì chỉ in hệ số.
- Đơn thức có số mũ bằng 1 thì không in số mũ.

Ví dụ:

Input	Output
7	
2 15	
-1 10	
0 8	Đa thức vừa nhập là: 2x ¹⁵ -x ¹⁰ +x ⁵ +6x ² -8.5x-10.52
1 5	Với x=2, giá trị đa thức là: 64540.48
6 2	
-8.5 1	
-10.52 0	
2	
4	
0 20	
-10 6	Đa thức vừa nhập là: -10x ⁶ +x ⁵ -1
1 5	Với x=3.55, giá trị đa thức là: -19452.85
-1 0	
3.55	
3	
0 4	Đa thức vừa nhập là: 0
0 3	Với x=10, giá trị đa thức là: 0.00
0 1	
10	

T003.Q29.BTT3.Lop1

10 problems with a total score of 10

#	Problem	Score
1	T003.Q29.Tuan3.TailOp.Cau1-DSLK	1
2	T003.Q29.Tuan3.TailOp.Cau2-DSLK	1
3	T003.Q29.Tuan3.TailOp.Cau3-STACK	1
4	T003.Q29.Tuan3.TailOp.Cau4-STACK	1
5	T003.Q29.Tuan3.TailOp.Cau5-ShellSort(template)	1
6	T003.Q29.Tuan3.TailOp.Cau6-HeapSort(template)	1
7	T003.Q29.Tuan3.TailOp.Cau7-QuickSort(template)	1
8	T003.Q29.Tuan3.TailOp.Cau8-BinaryInsertionSort(template)	1
9	T003.Q29.Tuan3.TailOp.Cau9-RadixSort(template)	1
10	T003.Q29.Tuan3.TailOp.Cau10-InterchangeSort(template)	1

Shell Sort

Bài toán: Thuật toán sắp xếp Shell (Shell Sort).

Shell Sort cải tiến từ insertion sort bằng cách so sánh các phần tử cách nhau một khoảng gap . Khoảng cách giảm dần (thường là chia 2) đến khi $gap = 1$.

Yêu cầu: Hãy cài đặt Shell Sort tăng dần cho mảng có N phần tử. Trong quá trình chạy thuật toán, mỗi khi mảng thay đổi (dịch chuyển hoặc chèn lại phần tử), hãy in ra cấu hình hiện tại của mảng.

Input:

- Dòng đầu tiên là số nguyên N dương ($0 < N < 20$).
- Dòng tiếp theo chứa N số nguyên Ai là các phần tử của mảng.

Output:

- In ra các dòng cấu hình của mảng sau mỗi lần mảng thay đổi trong Shell Sort.

Ví dụ:

INPUT	OUTPUT
	49920 15837 49920 3601
	14483 15837 49920 3601
	14483 15837 49920 15837
4	14483 3601 49920 15837
49920 15837 14483 3601	14483 14483 49920 15837
	3601 14483 49920 15837
	3601 14483 49920 49920
	3601 14483 15837 49920

T003.Q29.BTT3.Lop1

10 problems with a total score of 10

#	Problem	Score
1	T003.Q29.Tuan3.Tailop.Cau1-DSLK	1
2	T003.Q29.Tuan3.Tailop.Cau2-DSLK	1
3	T003.Q29.Tuan3.Tailop.Cau3-STACK	1
4	T003.Q29.Tuan3.Tailop.Cau4-STACK	1
5	T003.Q29.Tuan3.Tailop.Cau5-ShellSort(template)	1
6	T003.Q29.Tuan3.Tailop.Cau6-HeapSort(template)	1
7	T003.Q29.Tuan3.Tailop.Cau7-QuickSort(template)	1
8	T003.Q29.Tuan3.Tailop.Cau8-BinaryInsertionSort(template)	1
9	T003.Q29.Tuan3.Tailop.Cau9-RadixSort(template)	1
10	T003.Q29.Tuan3.Tailop.Cau10-InterchangeSort(template)	1

Heap Sort

Bài toán: Thuật toán sắp xếp vun đống (Heap Sort).

Heap Sort sử dụng cấu trúc *max-heap* để đưa phần tử lớn nhất về cuối mảng, sau đó tiếp tục vun đống lại cho phần mảng còn lại đến khi mảng được sắp xếp tăng dần.

Yêu cầu: Hãy cài đặt Heap Sort tăng dần cho mảng có N phần tử. Trong quá trình chạy thuật toán, mỗi khi mảng thay đổi do thao tác đổi chỗ (*swap*), hãy in ra cấu hình hiện tại của mảng.

Input:

- Dòng đầu tiên là số nguyên N dương ($0 < N < 20$).
- Dòng tiếp theo chứa N số nguyên Ai là các phần tử của mảng.

Output:

- In ra các dòng cấu hình của mảng sau mỗi lần mảng thay đổi trong Heap Sort.

Ví dụ:

INPUT	OUTPUT
	3601 15837 14483 49920
4	15837 3601 14483 49920
49920 15837 14483 3601	14483 3601 15837 49920
	3601 14483 15837 49920

T003.Q29.BTT3.Lop1

10 problems with a total score of 10

#	Problem	Score
1	T003.Q29.Tuan3.Tailop.Cau1-DSLK	1
2	T003.Q29.Tuan3.Tailop.Cau2-DSLK	1
3	T003.Q29.Tuan3.Tailop.Cau3-STACK	1
4	T003.Q29.Tuan3.Tailop.Cau4-STACK	1
5	T003.Q29.Tuan3.Tailop.Cau5-ShellSort(template)	1
6	T003.Q29.Tuan3.Tailop.Cau6-HeapSort(template)	1
7	T003.Q29.Tuan3.Tailop.Cau7-QuickSort(template)	1
8	T003.Q29.Tuan3.Tailop.Cau8-BinaryInsertionSort(template)	1
9	T003.Q29.Tuan3.Tailop.Cau9-RadixSort(template)	1
10	T003.Q29.Tuan3.Tailop.Cau10-InterchangeSort(template)	1

Quick Sort

Bài toán: Thuật toán sắp xếp nhanh (Quick Sort).

Quick Sort chọn một phần tử làm *pivot*, phân hoạch mảng thành hai phần (nhỏ hơn hoặc bằng pivot ở bên trái, lớn hơn pivot ở bên phải), sau đó đệ quy sắp xếp từng phần.

Yêu cầu: Hãy cài đặt Quick Sort tăng dần cho mảng có N phần tử. Trong quá trình chạy thuật toán, mỗi khi xảy ra thao tác đổi chỗ (*swap*) làm mảng thay đổi, hãy in ra cấu hình hiện tại của mảng.

Input:

- Dòng đầu tiên là số nguyên N dương ($0 < N < 20$).
- Dòng tiếp theo chứa N số nguyên Ai là các phần tử của mảng.

Output:

- In ra các dòng cấu hình của mảng sau mỗi lần mảng thay đổi do thao tác swap trong Quick Sort.

Ví dụ:

INPUT	OUTPUT
	3601 15837 14483 49920
4	3601 15837 14483 49920
49920 15837 14483 3601	3601 14483 15837 49920

T003.Q29.BTT3.Lop1

10 problems with a total score of 10

#	Problem	Score
1	T003.Q29.Tuan3.Tailop.Cau1-DSLK	1
2	T003.Q29.Tuan3.Tailop.Cau2-DSLK	1
3	T003.Q29.Tuan3.Tailop.Cau3-STACK	1
4	T003.Q29.Tuan3.Tailop.Cau4-STACK	1
5	T003.Q29.Tuan3.Tailop.Cau5-ShellSort(template)	1
6	T003.Q29.Tuan3.Tailop.Cau6-HeapSort(template)	1
7	T003.Q29.Tuan3.Tailop.Cau7-QuickSort(template)	1
8	T003.Q29.Tuan3.Tailop.Cau8-BinaryInsertionSort(template)	1
9	T003.Q29.Tuan3.Tailop.Cau9-RadixSort(template)	1
10	T003.Q29.Tuan3.Tailop.Cau10-InterchangeSort(template)	1

Binary Insertion Sort

Bài toán: Thuật toán sắp xếp chèn nhị phân (Binary Insertion Sort).

Binary Insertion Sort dùng tìm kiếm nhị phân để xác định vị trí chèn của phần tử hiện tại trong đoạn đã sắp xếp, sau đó dịch các phần tử để chèn đúng vị trí.

Yêu cầu: Hãy cài đặt Binary Insertion Sort tăng dần cho mảng có N phần tử. Trong quá trình chạy thuật toán, mỗi khi mảng thay đổi (dịch phần tử hoặc chèn phần tử), hãy in ra cấu hình hiện tại của mảng.

Input:

- Dòng đầu tiên là số nguyên N dương ($0 < N < 20$).
- Dòng tiếp theo chứa N số nguyên A*i* là các phần tử của mảng.

Output:

- In ra các dòng cấu hình của mảng sau mỗi lần mảng thay đổi trong Binary Insertion Sort.

Ví dụ:

INPUT	OUTPUT
	49920 49920 14483 3601
	15837 49920 14483 3601
	15837 49920 49920 3601
	15837 15837 49920 3601
4	14483 15837 49920 3601
49920 15837 14483 3601	14483 15837 49920 49920
	14483 15837 15837 49920
	14483 14483 15837 49920
	3601 14483 15837 49920

T003.Q29.BTT3.Lop1

10 problems with a total score of 10

#	Problem	Score
1	T003.Q29.Tuan3.Tailop.Cau1-DSLK	1
2	T003.Q29.Tuan3.Tailop.Cau2-DSLK	1
3	T003.Q29.Tuan3.Tailop.Cau3-STACK	1
4	T003.Q29.Tuan3.Tailop.Cau4-STACK	1
5	T003.Q29.Tuan3.Tailop.Cau5-ShellSort(template)	1
6	T003.Q29.Tuan3.Tailop.Cau6-HeapSort(template)	1
7	T003.Q29.Tuan3.Tailop.Cau7-QuickSort(template)	1
8	T003.Q29.Tuan3.Tailop.Cau8-BinaryInsertionSort(template)	1
9	T003.Q29.Tuan3.Tailop.Cau9-RadixSort(template)	1
10	T003.Q29.Tuan3.Tailop.Cau10-InterchangeSort(template)	1

Radix Sort

Bài toán: Thuật toán sắp xếp cơ sở (Radix Sort) cho số nguyên không âm.

Radix Sort sắp xếp theo từng chữ số từ phải sang trái (LSD): hàng đơn vị, chục, trăm, ... Mỗi lượt dùng counting sort ổn định theo chữ số đang xét.

Yêu cầu: Hãy cài đặt Radix Sort tăng dần cho mảng có N phần tử. Sau mỗi lượt sắp xếp theo một chữ số, nếu mảng thay đổi thì in ra cấu hình hiện tại của mảng.

Input:

- Dòng đầu tiên là số nguyên N dương ($0 < N < 20$).
- Dòng tiếp theo chứa N số nguyên không âm A*i* là các phần tử của mảng.

Output:

- In ra các dòng cấu hình của mảng sau mỗi lượt theo chữ số nếu mảng có thay đổi.

Ví dụ:

INPUT	OUTPUT
	49920 3601 14483 15837
4	3601 49920 15837 14483
49920 15837 14483 3601	14483 3601 15837 49920
	3601 14483 15837 49920

T003.Q29.BTT3.Lop1

10 problems with a total score of 10

#	Problem	Score
1	T003.Q29.Tuan3.Tailop.Cau1-DSLK	1
2	T003.Q29.Tuan3.Tailop.Cau2-DSLK	1
3	T003.Q29.Tuan3.Tailop.Cau3-STACK	1
4	T003.Q29.Tuan3.Tailop.Cau4-STACK	1
5	T003.Q29.Tuan3.Tailop.Cau5-ShellSort(template)	1
6	T003.Q29.Tuan3.Tailop.Cau6-HeapSort(template)	1
7	T003.Q29.Tuan3.Tailop.Cau7-QuickSort(template)	1
8	T003.Q29.Tuan3.Tailop.Cau8-BinaryInsertionSort(template)	1
9	T003.Q29.Tuan3.Tailop.Cau9-RadixSort(template)	1
10	T003.Q29.Tuan3.Tailop.Cau10-InterchangeSort(template)	1

Interchange Sort

Bài toán: Thuật toán sắp xếp đổi chỗ trực tiếp (Interchange Sort).

Interchange Sort duyệt từng phần tử $a[i]$ và so sánh với các phần tử đứng sau $a[j]$ ($j > i$). Nếu $a[i] > a[j]$ thì đổi chỗ ngay.

Yêu cầu: Hãy cài đặt Interchange Sort tăng dần cho mảng có N phần tử. Trong quá trình chạy thuật toán, mỗi khi xảy ra thao tác đổi chỗ (swap) làm mảng thay đổi, hãy in ra cấu hình hiện tại của mảng.

Input:

- Dòng đầu tiên là số nguyên N dương ($0 < N < 20$).
- Dòng tiếp theo chứa N số nguyên A*i* là các phần tử của mảng.

Output:

- In ra các dòng cấu hình của mảng sau mỗi lần swap trong Interchange Sort.

Ví dụ:

INPUT	OUTPUT
	15837 49920 14483 3601
	14483 49920 15837 3601
4	3601 49920 15837 14483
49920 15837 14483 3601	3601 15837 49920 14483
	3601 14483 49920 15837
	3601 14483 15837 49920

T003.Q29.BTT3.Lop1

10 problems with a total score of 10

#	Problem	Score
1	T003.Q29.Tuan3.Tailop.Cau1-DSLK	1
2	T003.Q29.Tuan3.Tailop.Cau2-DSLK	1
3	T003.Q29.Tuan3.Tailop.Cau3-STACK	1
4	T003.Q29.Tuan3.Tailop.Cau4-STACK	1
5	T003.Q29.Tuan3.Tailop.Cau5-ShellSort(template)	1
6	T003.Q29.Tuan3.Tailop.Cau6-HeapSort(template)	1
7	T003.Q29.Tuan3.Tailop.Cau7-QuickSort(template)	1
8	T003.Q29.Tuan3.Tailop.Cau8-BinaryInsertionSort(template)	1
9	T003.Q29.Tuan3.Tailop.Cau9-RadixSort(template)	1
10	T003.Q29.Tuan3.Tailop.Cau10-InterchangeSort(template)	1